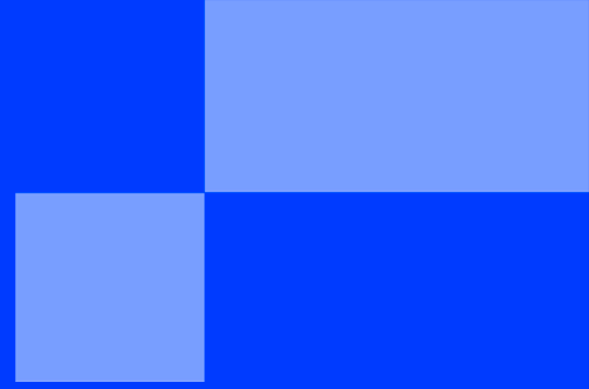




SUMMER SCHOOL

< PART 5 BOOTSTRAP - FULL STACK
INTEGRATION />

SUMMER SCHOOL

< WEB DEVELOPMENT: TYING IT ALL TOGETHER />

"We become what we behold. We shape our tools, and thereafter our tools shape us."
— **Marshall McLuhan**

Introduction

You have reached the final stage. You know how to build a Front-End (React) and a Back-End (Express). But right now, they are isolated.

In this bootstrap, you will learn three crucial skills for modern web development:

- ✓ **The NPM Ecosystem & Tailwind CSS:** Installing community tools to build beautiful User Interfaces instantly.
- ✓ **Data Fetching (CRUD):** Using the `fetch` API to Create, Read, Update, and Delete data on a server.
- ✓ **Routing:** Using `react-router-dom` to create a Single Page Application (SPA) with multiple pages without ever reloading the browser.
- ✓ **json-server:** A fake API server that allows you to practice your Front-End skills without having to build a Back-End first.

To practice, we will build a **Live Message Board**. It will have two distinct pages: a Dashboard to view messages, and a dedicated Publishing page to draft new ones.



What is json-server doing exactly?

When your React app calls `fetch('http://localhost:3001/posts')`, json-server intercepts that request and returns the data from your `db.json` file, just like a real Express server would. When you DELETE a post, json-server actually removes it from `db.json`. Your changes **persist** across reloads!

Step 1: NPM and The Library Ecosystem

When you installed Node.js, it came with **NPM (Node Package Manager)**, a massive online marketplace of free code libraries. We will use NPM to install Tailwind CSS, React Router, and our fake API server.

Create your sandbox and install our tools:

```
npm create vite@latest fullstack-sandbox -- --template react
cd fullstack-sandbox
npm install
```

Now, install **Tailwind CSS** (for styling), **React Router** (for page navigation), and **json-server** (for our local fake API):

```
npm install -D tailwindcss@3 postcss autoprefixer json-server
npm install react-router-dom
npx tailwindcss init -p
```

Configure Tailwind by opening `tailwind.config.js` and replacing its content:

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.js,ts,jsx,tsx",
  ],
  theme: { extend: {} },
  plugins: [],
}
```

Open `src/index.css`, delete everything, and inject the Tailwind tools:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Step 2: Setting up the Fake API with json-server

Before touching React, we need a real API to talk to. Instead of building an Express server from scratch, we will use **json-server**: a tool that reads a simple JSON file and instantly exposes it as a full REST API, with GET, POST, DELETE and more, all working out of the box.

Create a file named `db.json` at the root of your project:

```
{
  "posts": [
    { "id": 1, "title": "Welcome to the board", "body": "This is your first post. Feel free to delete it!" },
    { "id": 2, "title": "React is awesome", "body": "Building UIs with components makes everything cleaner." },
    { "id": 3, "title": "Tailwind saves time", "body": "Utility-first CSS means you never leave your JSX." },
    { "id": 4, "title": "What is a SPA?", "body": "A Single Page Application loads once and updates the view without reloading." },
    { "id": 5, "title": "Express on the backend", "body": "Node.js + Express lets you build REST APIs in minutes." },
    { "id": 6, "title": "CRUD in a nutshell", "body": "Create, Read, Update, Delete, the four operations of any data-driven app." }
  ]
}
```

Now open `package.json` and add a `server` script:

```
"scripts": {
  "dev": "vite",
  "server": "json-server --watch db.json --port 3001"
}
```



Why two terminals?

Your React app (Vite) and your API (json-server) are two separate processes. From now on, every time you work on this project, you will open **two terminals**: one for `npm run dev`, one for `npm run server`.

Open two terminals and run:

```
# Terminal 1 - React app
npm run dev

# Terminal 2 - Fake API
npm run server
```

Your API is now live at `http://localhost:3001/posts`. Open it in your browser to see your data!

Step 3: Setting up the Router

React is designed to be a Single Page Application. To fake the existence of multiple pages (like `/home` and `/publish`), we use a Router. It watches the URL in the browser and swaps the components on the screen instantly.

Open `src/main.jsx` and wrap your `<App />` inside a `<BrowserRouter>`:

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom' // 1. Import the Router
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    { /* 2. Wrap the App */ }
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)
```

Now, let's define our layout and our routes. Open `src/App.jsx` and replace everything with:

```
import { Routes, Route, Link } from 'react-router-dom';
import Board from './pages/Board';
import Publish from './pages/Publish';

function App() {
  return (
    <div className="min-h-screen bg-slate-50 font-sans text-slate-800">

      {/* GLOBAL NAVIGATION BAR */}
      <nav className="bg-white shadow-sm border-b border-slate-200 px-8 py-4 flex justify-between items-center">
        <div className="text-xl font-extrabold text-slate-900">
          Message<span className="text-indigo-600">Board</span>
        </div>
        <div className="space-x-4">
          {/* We use <Link> instead of <a> to navigate WITHOUT reloading the page */}
          <Link to="/" className="text-slate-600 hover:text-indigo-600 font-medium">View
            Posts</Link>
          <Link to="/publish" className="bg-indigo-600 text-white px-4 py-2 rounded-lg font-bold
            hover:bg-indigo-700 transition">Write a Post</Link>
        </div>
      </nav>

      {/* ROUTER LOGIC: Which page should be displayed? */}
      <main className="p-8">
        <Routes>
          <Route path="/" element={<Board />} />
          <Route path="/publish" element={<Publish />} />
        </Routes>
      </main>

    </div>
  );
}

export default App;
```

Start your server with `npm run dev`. Your app will currently crash because the `Board` and `Publish` pages do not exist yet. Let's build them!

Step 4: The Board Page (GET & DELETE)

Create a new folder named `src/pages/`. Inside it, create a file named `Board.jsx`. This page will fetch the data from your local API when it loads, and display it in a grid.

```
// src/pages/Board.jsx
import { useState, useEffect } from 'react';

const API_URL = 'http://localhost:3001';

function Board() {
  const [posts, setPosts] = useState([]);

  // 1. READ (GET Data)
  useEffect(() => {
    fetch(`${API_URL}/posts`)
      .then(response => response.json())
      .then(data => setPosts(data));
  }, []);

  // 2. DELETE
  const handleDelete = (id) => {
    fetch(`${API_URL}/posts/${id}`, { method: 'DELETE' })
      .then(() => {
        // Remove from the screen by filtering out the deleted ID
        setPosts(posts.filter(post => post.id !== id));
      });
  };

  return (
    <div className="max-w-7xl mx-auto">
      <h1 className="text-3xl font-extrabold mb-8">Recent Discussions</h1>

      { /* 3-Column Grid */ }
      <div className="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3 gap-6">
        {posts.map((post) => (
          <div key={post.id} className="bg-white rounded-2xl p-6 shadow-sm border
            border-slate-200 flex flex-col justify-between">
            <div>
              <h2 className="text-lg font-bold text-slate-900 mb-2 capitalize">{post.title}</h2>
              <p className="text-slate-500 text-sm mb-6">{post.body}</p>
            </div>
            <button onClick={() => handleDelete(post.id)} className="text-red-500 text-sm
              font-bold bg-red-50 hover:bg-red-100 px-4 py-2 rounded-lg self-start transition">
              Delete Post
            </button>
          </div>
        ))}
      </div>
    </div>
  );
}

export default Board;
```

Step 5: The Publish Page (POST)

Inside `src/pages/`, create a second file named `Publish.jsx`.

This page contains a form. When submitted, it will send the data to the API, and then automatically redirect the user back to the Board page!

```
// src/pages/Publish.jsx
import { useState } from 'react';
import { useNavigate } from 'react-router-dom'; // Allows us to change pages via code

const API_URL = 'http://localhost:3001';

function Publish() {
  const [title, setTitle] = useState('');
  const [body, setBody] = useState('');
  const navigate = useNavigate(); // Hook to redirect the user

  // CREATE (POST Data)
  const handlePublish = (e) => {
    e.preventDefault();

    const newPostData = { title: title, body: body };

    fetch(`${API_URL}/posts`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(newPostData)
    })
    .then(response => response.json())
    .then(createdPost => {
      console.log("Successfully published:", createdPost);
      // After successful creation, automatically send the user back to the Home page!
      navigate('/');
    });
  };

  return (
    <div className="max-w-2xl mx-auto bg-white p-8 rounded-2xl shadow-sm border border-slate-200">
      <h1 className="text-3xl font-extrabold mb-8">Draft a New Message</h1>

      <form onSubmit={handlePublish} className="flex flex-col gap-6">
        <div>
          <label className="block text-sm font-bold text-slate-700 mb-2">Message Title</label>
          <input
            type="text"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
            required
            className="w-full px-4 py-3 rounded-xl border border-slate-300 focus:ring-2 focus:ring-indigo-500 outline-none"
          />
        </div>
      </form>
    </div>
  );
}
```



```

    <div>
      <label className="block text-sm font-bold text-slate-700 mb-2">Message Content</label>
      <textarea
        rows="5"
        value={body}
        onChange={(e) => setBody(e.target.value)}
        required
        className="w-full px-4 py-3 rounded-xl border border-slate-300 focus:ring-2
        focus:ring-indigo-500 outline-none resize-none"
      ></textarea>
    </div>

    <button type="submit" className="bg-indigo-600 text-white font-bold px-6 py-4 rounded-xl
    hover:bg-indigo-700 transition mt-4">
      Publish Message
    </button>
  </form>
</div>
);
}

export default Publish;

```

Action: Open your browser! Go to the "Write a Post" page, fill out a title and a message, and hit Publish. Watch as it redirects you back to the board and your new post is right there. Reload the page: it's still there. Open `db.json`: it was saved!

Conclusion

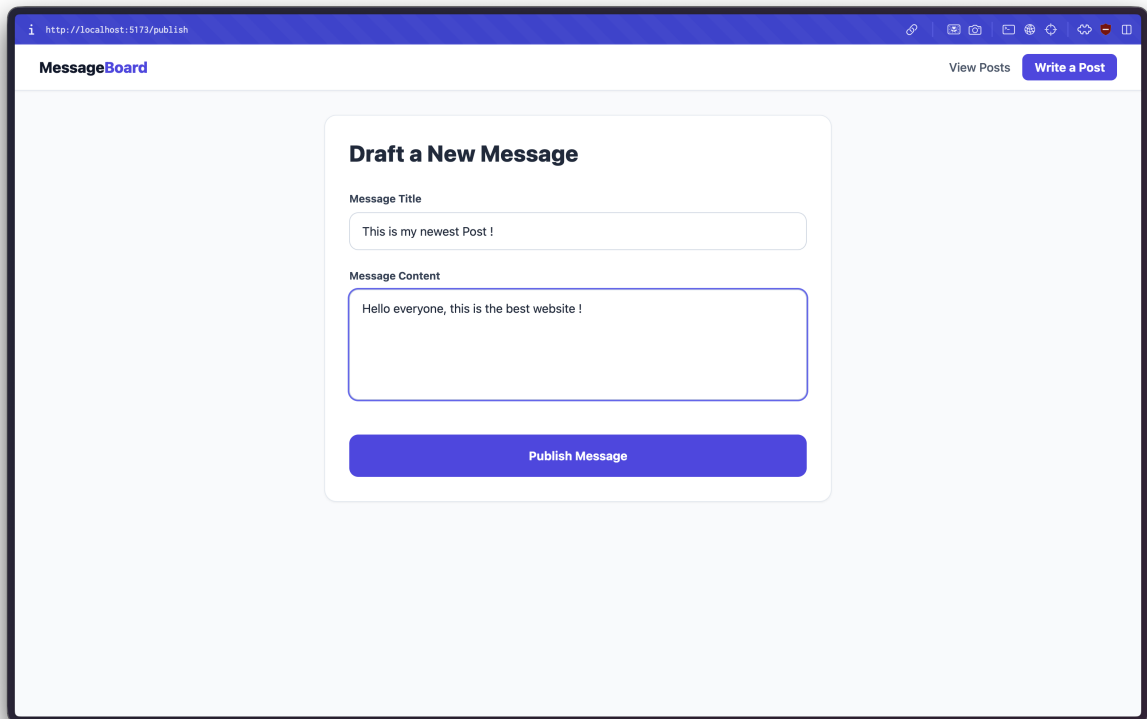
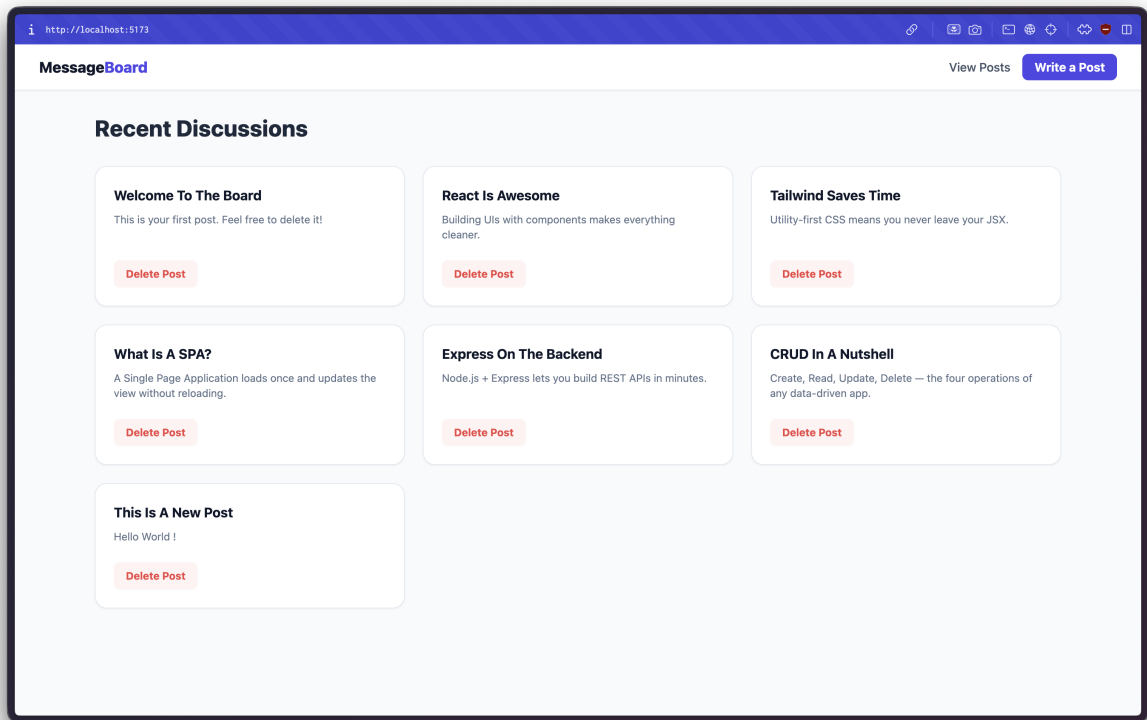
You are now a Full-Stack developer capable of building modern Single Page Applications.

Close your sandbox, open your main `newspaper-react` project.

1. Install **Tailwind CSS** and **React Router**.
2. Start your `newspaper-backend` server.
3. Build a real digital newspaper with a reading page and a publishing dashboard!



The key insight: The only difference between this sandbox and the final project is the API URL. Here you used `http://localhost:3001`. In the final project, you will point your `fetch` calls to your own Express backend, that is the entire bridge between Front-End and Back-End.



v1

{EPITECH}